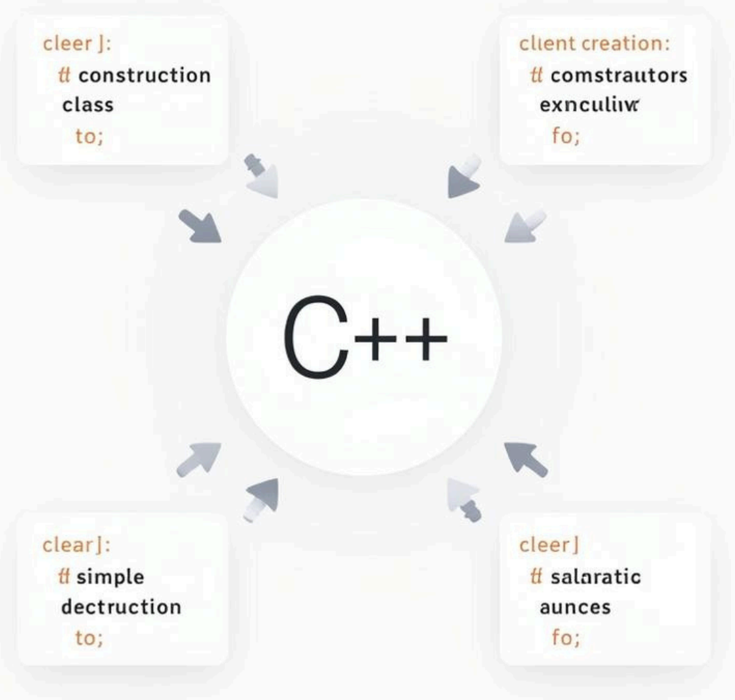
Introduction to Constructors & Destructors

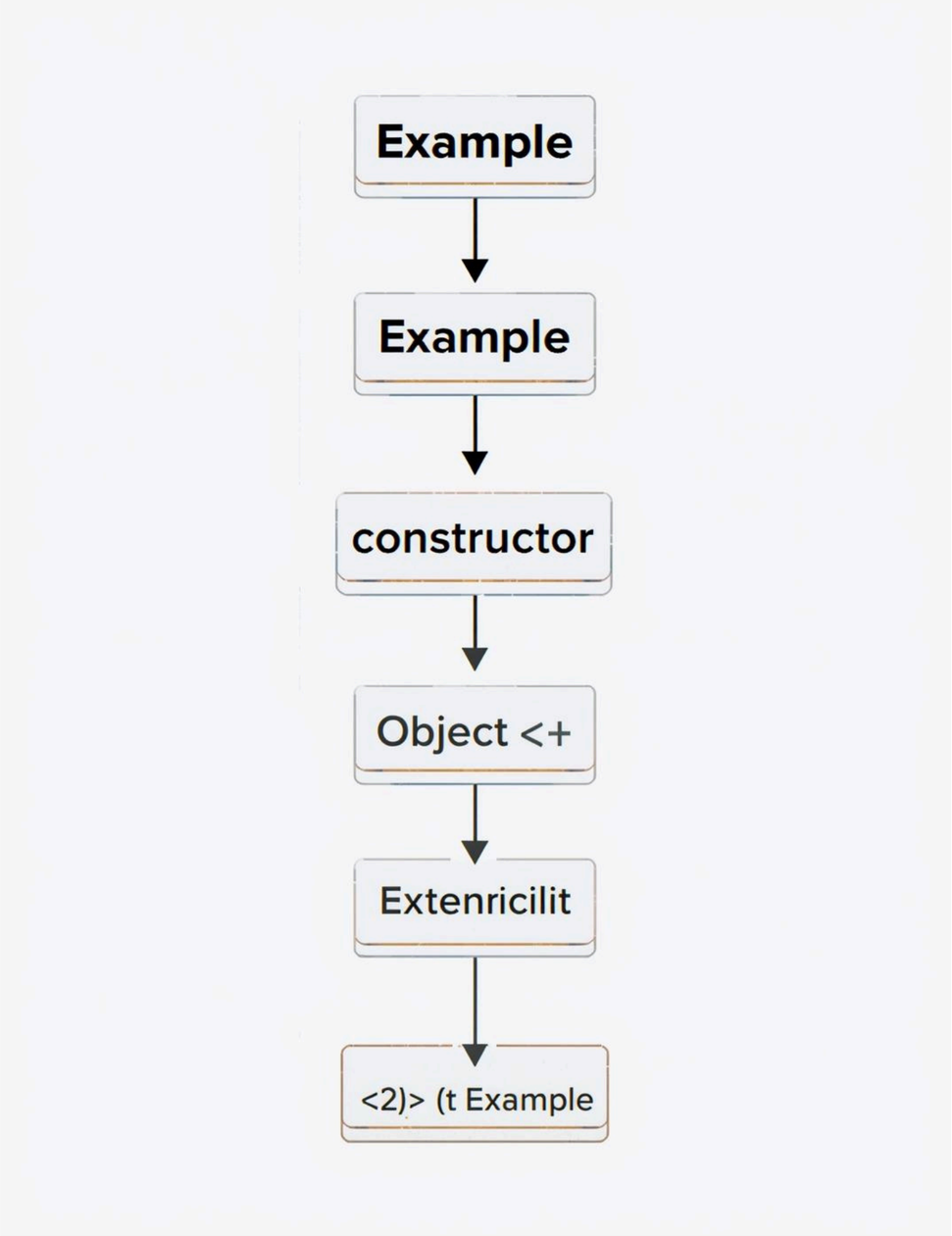


Presented by Zainab Furqan

Understanding Constructors

Definition and Purpose

A constructor is a special class function that initializes objects, automatically invoked when an object is created, allowing for optimal setup and resource allocation.



Default Constructor

Syntax and Usage

A **default constructor** initializes an object without parameters. It is automatically called when an object is created, ensuring member variables have default values for predictable behavior.

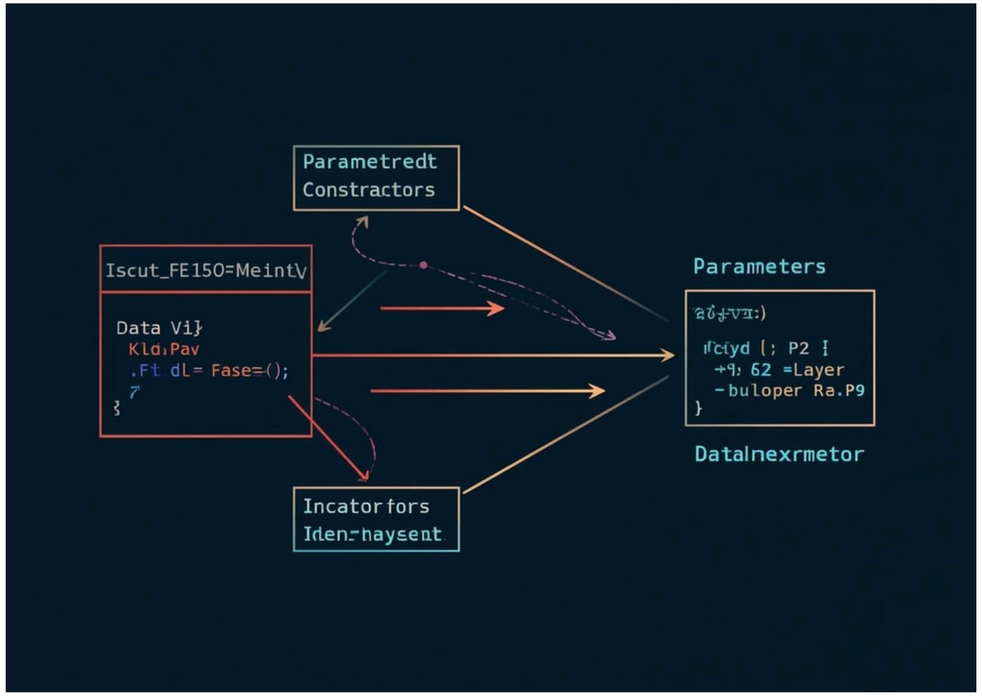
```
#include <iostream>
using namespace std;
class construct {
public:
 int a, b;
 construct(){
 a = 10;
 b = 20;
 }
};
int main(){
 construct c;
 cout << "a: " << c.a << endl
 << "b: " << c.b;
 return 1;
}
```

Parameterized Constructors

Syntax and Practical Usage

A parameterized constructor allows custom **initialization** of objects by accepting arguments. This enables flexible object creation, adapting to different contexts and enhancing code efficiency.

- 📝 This type of constructor accepts arguments inside its parenthesis.
- 📝 Is called when creating objects by passing relevant arguments.
- 📝 These arguments help initialize an object when it is created.
- 📝 To create a parameterized constructor, simply add parameters to it the way you
- would to any other function.
- 📝 When you define the constructor's body, use the parameters to initialize the
- object.



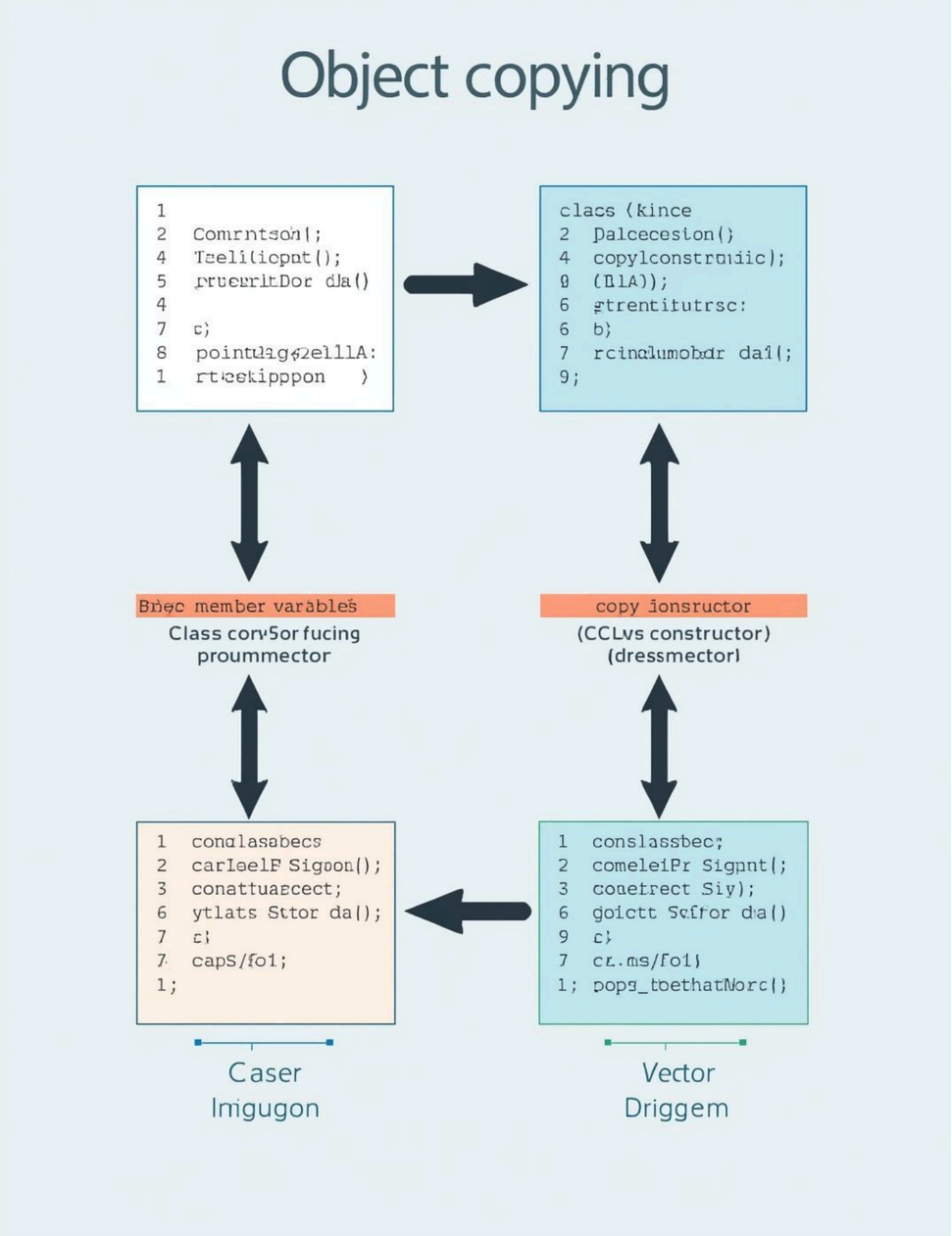

```
class Point {
private:
 int x, y;
public:
 Point(int x1, int y1){
 x = x1;
 y = y1;
 }
 int getX(){
 return x;
 }
 int getY(){
 return y;
 }
};
```

Copy Constructor

Understanding Object Copying

A copy constructor creates a new object as a **copy** of an existing object, ensuring that all data members are accurately duplicated, especially in dynamic memory scenarios.

```
class Example {
 int a, b;
public:
 Example(int x, int y) {
 a = x;
 b = y;
 cout << "\nIm Constructor";
 }
 Example(const Example& obj) {
 a = obj.a;
 b = obj.b;
 cout << "\nIm Copy Constructor";
 }
 void Display() {
 cout << "\nValues :" << a << "\t" << b;
 }
};
int main() {
 Example Object(10, 20);
 Example Object2(Object);
 Example Object3 = Object;
 Object.Display();
 Object2.Display();
 Object3.Display();
 return 0;
```



Practice Questions

Create a class Student with name and age. Initialize values using a default constructor and display them.

```
class Student {
public:
    string name;
    int age;

    Student() {
        name = "Ali";
        age = 20;
    }
};
```

Practice Questions

Create a class Rectangle that calculates area using length and width passed to constructor.

```
class Rectangle {
    int length, width;
public:
    Rectangle(int l, int w) {
        length = l;
        width = w;
    }
    int area() {
        return length * width;
    }
};
```

Practice Questions

Create a class that stores an integer using dynamic memory allocation and ensure that a deep copy is performed.

```
#include <iostream>
using namespace std;

class Number {
    int *value;
public:
    // Parameterized constructor
    Number(int v) {
        value = new int;
        *value = v;
    }

    // Copy constructor (deep copy)
    Number(const Number &obj) {
        value = new int;
        *value = *(obj.value);
    }

    void display() {
        cout << "Value: " << *value << endl;
    }

    // Destructor
    ~Number() {
        delete value;
    }
};
```

```
int main() {
Number n1(10);
Number n2 = n1; // Copy constructor called
n1.display();
n2.display();
return 0;
}
```

What is a Destructor?

Understanding Destructor Purpose

A destructor is a special member function invoked when an object goes out of scope, ensuring proper resource deallocation and preventing memory leaks in C++.

Destructor does not have arguments. It has no return type not even void. An object of a
class with a Destructor cannot become a member of the union. A destructor should be
declared in the public section of the class. The programmer cannot access the address of
destructor.

```
class  Person {
  public:
    Person(){ // constructor
      // code
    }
    ~Person(){ //destructor
      // code
    }
};
```

Destructor Syntax

Understanding Destructors

A destructor is a special member function executed when an object goes out of scope, allowing for resource cleanup and memory deallocation in C++.

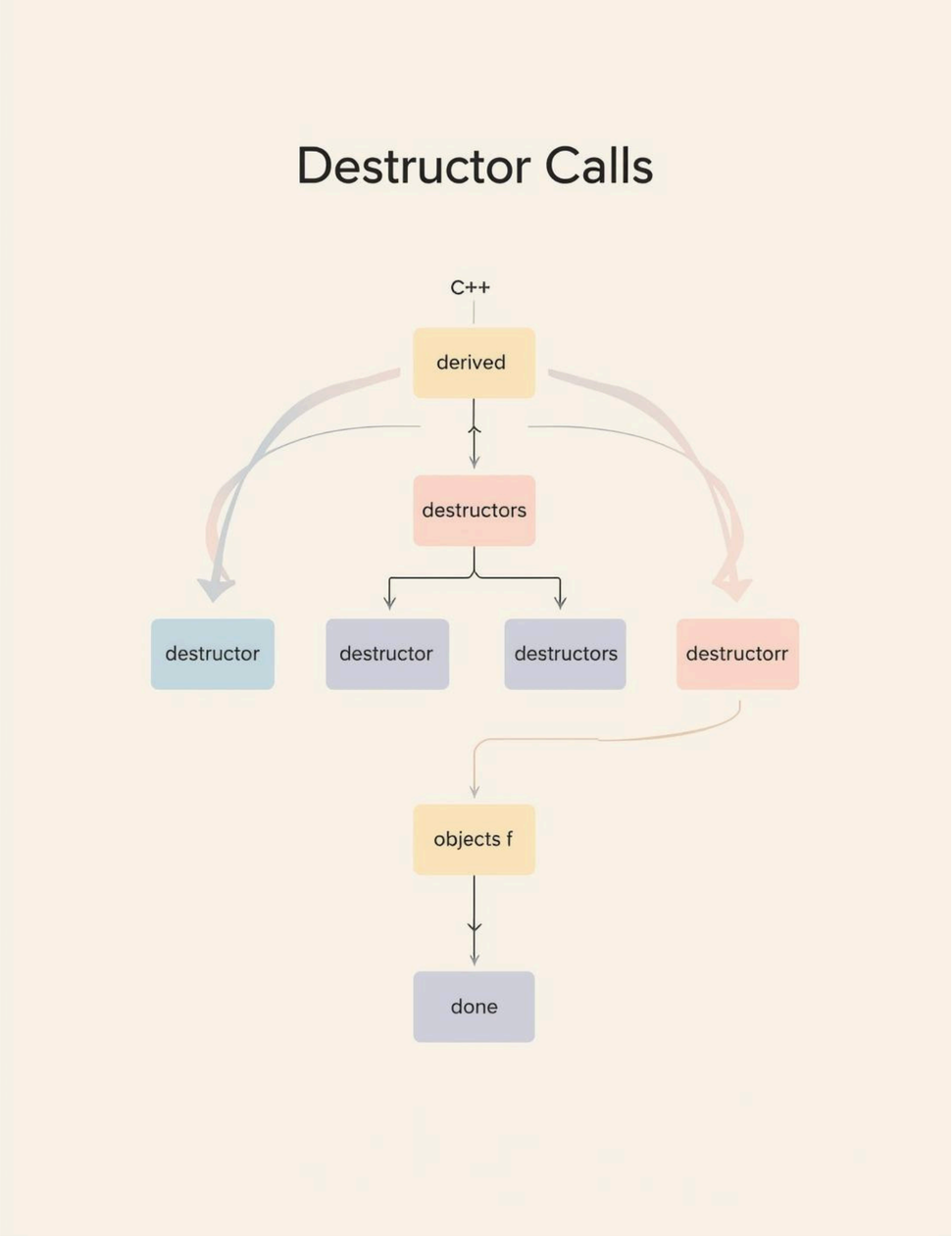
```
#include <iostream>
using namespace std;
class HelloWorld{
public:
 HelloWorld(){
 cout<<"Constructor is called"<<endl;
 }
 ~HelloWorld(){
 cout<<"Destructor is called"<<endl;
 }
 void display(){
 cout<<"Hello World!"<<endl;
 }
};
int main(){
 HelloWorld obj;
 obj.display();
 return 0;
}
```

Order of Destructor Calls

Understanding Destructor Sequence

In C++, destructors are called in the reverse order of constructors. This ensures proper resource management and cleanup when an object's lifetime ends, preventing memory leaks.

Inheritance in C++ involves calling the base class constructor before the derived class constructor, ensuring proper initialization of class members and resources throughout the hierarchy.



Predict the Output

Predict the output of the following program and explain the sequence of constructor and destructor calls.

```
#include <iostream>
using namespace std;

class Test {
public:
    Test() {
        cout << "Constructor" << endl;
    }
    ~Test() {
        cout << "Destructor" << endl;
    }
};

int main() {
    Test t1, t2;
    return 0;
}
```

Final Questions

1. Create a Student class that:
- Uses a default constructor to initialize the name as "Unknown".
- Uses a parameterized constructor to initialize the name with a user-provided value.
- Uses a copy constructor to create a deep copy of another Student object.
- Uses a destructor to properly release dynamically allocated memory.
2. In the main() function:
- Create one student object using the default constructor.
- Create another student object using the parameterized constructor.
- Create a third student object using the copy constructor.
- Display all student names.
- Observe and explain the order of constructor and destructor calls.
3. Modify the copied object’s name and verify that the original object remains unchanged.

Questions or Comments?

We appreciate your attention and feedback today!